Zlecenie Projektu

1. Metadane i Powiązanie Programowe

Nazwa projektu: Projekt 4: Identyfikacja Aplikacji w Szyfrowanym Kanale HTTPS/TLS na Podstawie Rozmiaru Pakietów (k-NN) / Project 4: Encrypted Traffic Application Identification in HTTPS/TLS Using k-NN.
Acro-Dictionary (Glosariusz skrótów): k-NN (k-Nearest Neighbors) - algorytm k-najbliższych sąsiadów, HTTPS - Hypertext Transfer Protocol Secure, TLS - Transport Layer Security.
Powiązanie z materiałem teoretycznym: Wykorzystanie algorytmu uczenia maszynowego (k-NN) do klasyfikacji ruchu sieciowego. Optymalizacja będzie dotyczyć doboru hiperparametru "k" dla osiągnięcia najwyższej skuteczności z jak najmniejszym obciążeniem obliczeniowym.
Wymagania sprzętowe i programistyczne: Python 3.x, Scikit-learn (do modelu k-NN), biblioteka Scapy lub narzędzie Wireshark (do parsowania PCAP i ekstrakcji cech).

2. Kontekst Praktyczny i Scenariusz Biznesowy (Use Case)

"So What?" (Znaczenie problemu): Szyfrowanie komunikacji skutecznie chroni prywatność użytkowników, ale stwarza wyzwania dla administratorów sieci firmowych w zakresie blokowania nieautoryzowanych i potencjalnie niebezpiecznych aplikacji (Shadow IT) lub przecieków danych. Klasyfikacja poprzez metadane rozwiązuje ten problem bez łamania szyfrowania.
Scenariusz wdrożeniowy: Moduł monitorowania dla Centrum Operacji Bezpieczeństwa (SOC), weryfikujący czy pracownicy zużywają pasmo na streaming wideo czy przeglądanie stron WWW, nie naruszając jednocześnie poufności przesyłanych informacji.

3. Problem Badawczy i Hipoteza (Research Problem & Hypothesis)

Pytanie badawcze: Czy na podstawie prostej analizy wektorów (rozmiary pakietów, czasy ich przybycia) można z dużą precyzją rozpoznać konkretną klasę ruchu sieciowego w całkowicie zaszyfrowanym kanale TLS za pomocą algorytmu k-NN?
Testowalna hipoteza: Zastosowanie klasyfikatora k-NN dla metadanych ruchu pozwala poprawnie kategoryzować zachowanie aplikacji sieciowych z dokładnością (Accuracy) powyżej 90%.

4. Formalny Model Zagrożeń (Threat Model)

Profil Przeciwnika: Passive Network Eavesdropper (Pasywny obserwator sieci) - podsłuchuje ruch na poziomie łącza/sieci.
Interfejs i Dostęp: Ciphertext-Only (tylko szyfrogram). Atakujący ma dostęp wyłącznie do

nagłówków L3/L4 (IP/TCP) oraz wielkości payloadów bez możliwości odszyfrowania właściwych treści.
Granice i Zasada Kerckhoffsa: Kod i architektura klasyfikatora są w pełni znane, ewaluacja następuje w typowym budżecie obliczeniowym standardowego sprzętu komputerowego.

5. Dane i Bazy do Analizy (Datasets & Input Data)

Źródło danych: UC Davis QUIC & TLS Traffic Dataset – https://github.com/ucdavis/quic-data
Charakterystyka danych: Pliki w formacie PCAP. Wymiarowość dla modelu zostaje uproszczona do 2 głównych cech (features): sekwencje rozmiarów pakietów (w bajtach) oraz ich inter-arrival times. Ilość danych można zredukować do małej, łatwej do przetworzenia próbki (np. kilka tysięcy flowów).

6. Instrukcja Wykonawcza Krok po Kroku (Execution Plan)

Podział danych: Zbiór podzielony metodą hold-out: 80% na zbiór treningowy, 20% na testowy.
Etapy prac:
- Faza 1: Przechwycenie ruchu i/lub pobranie datasetu PCAP, użycie narzędzi (Wireshark/Scapy) do analizy.
- Faza 2: Ekstrakcja prostych wektorów cech: wielkości pakietów i czasy dostarczenia.
- Faza 3: Implementacja i trening prostego algorytmu k-NN przy pomocy Scikit-Learn.
- Faza 4: Ewaluacja, wnioski i testy z paddingiem.

7. Grupa Kontrolna i Obowiązkowy Atak (Red Teaming & Baseline)

Eksperyment kontrolny (Baseline): Rozwiązanie k-NN będzie porównane do losowego przypisywania kategorii ruchu (Dummy Classifier) dla zademonstrowania nabytej zdolności do rozpoznawania wzorców.
Obowiązkowa próba przełamania: Zasymulowanie tzw. Paddingu (dokładania sztucznego narzutu danych). Do rozmiaru pakietów w zbiorze testowym należy dodać losową wartość z przedziału np. od 1 do 100 bajtów. Celem jest zbadanie o ile spadnie skuteczność k-NN, gdy ruch próbuje omijać detekcję poprzez zaciemnianie (obfuscation).

8. Kryterium Ukończenia i Mierniki Skuteczności (Metrics & Pass Criteria)

Ilościowe progi sukcesu: Projekt jest zdany, gdy na czystym zbiorze testowym dokładność (Accuracy) klasyfikacji rodzajów ruchu (wideo, audio, pliki itp.) będzie na poziomie przekraczającym 90% (Accuracy > 90%).
Mierniki wydajnościowe: Podstawowy pomiar narzutu: Czas wykonania predykcji. System powinien zwracać wynik w czasie umożliwiającym monitorowanie "w locie" (predykcja dla jednego wektora powinna być liczona w ułamkach sekund).

Końcowa forma: Aplikacja konsolowa, pozwalająca na klasyfikacje ruchu sieciowego w locie.

Literatura Naukowa (Opcjonalnie):

- Hayes, J., & Danezis, G. (2016). k-fingerprinting: A Robust Feature Based Website Fingerprinting Attack. USENIX Security 2016.

- Scapy Documentation (2024). Packet manipulation library for Python. https://scapy.net

- Biersack, E. et al. (2012). Data Analysis for Network CyberSecurity. Springer.